Assignment 8 — Pre-trained BERT for Sentiment Analysis

Deep Learning | Practical Report / Solution

BRACT's Vishwakarma Institute of Technology, Pune

Department: CSE-AI **Semester:** 5 **Academic Year:** 2026-27

Division: E **Batch:** 1 **Practical No.:** 8

Roll No.: 64 **PRN:** 12412320 **Name:** Samyak Shende

Problem Statement

Implement a pre-trained BERT model for sentiment analysis or text classification on a sample dataset.

Objective

Use a pretrained BERT sequence-classification model to classify sentiment in SST-2 validation examples, evaluate the predictions with standard classification metrics, visualize a confusion matrix, inspect high-confidence mistakes, and perform inference on a custom sentence.

Model and Dataset

The notebook uses **textattack/bert-base-uncased-SST-2** with the SST-2 validation split from the GLUE dataset. A reproducible sample of up to 500 validation examples is selected using seed 42.

Environment and Model Loading

The implementation installs Transformers, Datasets and Accelerate, imports PyTorch and scikit-learn utilities, selects CUDA when available, loads the tokenizer and pretrained sequence-classification model, and switches the model to evaluation mode.

Preprocessing and Batch Prediction

Text is tokenized in batches of 32 with padding and truncation to a maximum length of 128 tokens. Softmax probabilities are computed from the model logits. The predicted class is the class with the highest probability, and the positive-class probability is retained for analysis.

Evaluation Metrics

The notebook calculates accuracy, binary precision, recall, and F1-score using the ground-truth SST-2 labels and BERT predictions.

Confusion Matrix and Error Analysis

A confusion matrix is plotted with negative and positive labels. Misclassified examples are collected into a DataFrame, and a confidence score is calculated from the predicted class probability. The ten highest-confidence mistakes are displayed for review.

Custom Sentence Inference

The example sentence supplied in the practical sheet is: *"The story was slow, but the ending was wonderful."* The model returns a predicted sentiment and its positive probability.

Implementation Code

```
%pip install -q -U transformers datasets accelerate

import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import torch

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

MODEL_NAME = "textattack/bert-base-uncased-SST-2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME).to(device)
model.eval()
print("Device:", device)

dataset = load_dataset("nyu-mll/glue", "sst2", split="validation")
sample = dataset.shuffle(seed=SEED).select(range(min(500, len(dataset))))
display(pd.DataFrame(sample[:5]))

@torch.no_grad()
def predict_texts(texts, batch_size=32):
    predictions, positive_probabilities = [], []
    for start in range(0, len(texts), batch_size):
        batch = texts[start:start + batch_size]
        encoded = tokenizer(
            batch,
            padding=True,
            truncation=True,
            max_length=128,
            return_tensors="pt"
        ).to(device)
        probabilities = model(**encoded).logits.softmax(dim=-1)
        predictions.extend(probabilities.argmax(dim=-1).cpu().tolist())
        positive_probabilities.extend(probabilities[:, 1].cpu().tolist())
    return np.array(predictions), np.array(positive_probabilities)

texts = sample["sentence"]
y_true = np.array(sample["label"])
y_pred, positive_probability = predict_texts(texts)

accuracy = accuracy_score(y_true, y_pred)
precision, recall, f1, _ = precision_recall_fscore_support(
    y_true, y_pred, average="binary", zero_division=0
)

print(
    pd.Series({
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }).round(3)
)

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Purples",
    xticklabels=["negative", "positive"],
    yticklabels=["negative", "positive"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("BERT confusion matrix")
plt.show()

review = pd.DataFrame({
    "text": texts,
    "actual": y_true,
    "predicted": y_pred,
    "positive_probability": positive_probability,
})

mistakes = review[review.actual != review.predicted].copy()

mistakes["confidence"] = np.where(
    mistakes.predicted == 1,
    mistakes.positive_probability,
    1 - mistakes.positive_probability,
)

display(mistakes.sort_values("confidence", ascending=False).head(10))
```

```
custom_text = ["The story was slow, but the ending was wonderful."]

label, probability = predict_texts(custom_text)

print(
    custom_text[0],
    "->",
    ["negative", "positive"][label[0]],
    f"(positive probability={probability[0]:.3f})"
)
```

Results

The supplied practical sheet does not contain populated numerical output values. Therefore, no accuracy, precision, recall, F1-score, confusion-matrix counts, or custom inference probability is invented in this report. These values will be produced when the notebook is executed in the configured Python environment.

Conclusion

The experiment demonstrates the use of a pretrained BERT model for sentiment classification. The workflow covers dataset sampling, tokenization, batched inference, quantitative evaluation, confusion-matrix visualization, high-confidence error analysis, and custom sentence prediction. The supplied notebook provides a complete inference and evaluation pipeline while preserving the practical sheet's specified model and dataset.